

JavaScript応用

目次

- Progateからの追加構文
- 配列メソッド
- 非同期処理
- Viteを使ったReactの導入
- React入門
- React基礎
- CSSモジュール

はじめに

この教材ではReactを用いて簡単なフロントエンドが作成できることを目標としています。
Progate相当のJavaScriptが身についていることを前提としているため、まだできていない方はそちらを先に身に着けてください

Progateからの追加構文

分割代入（オブジェクト）

オブジェクトのプロパティを変数として取り出したいときには`let(const){プロパティ名...}=取りしたいオブジェクト`と書けます。

```
const getUser = () => {  
  return {id:1,name:"Tommy"}  
}  
  
const user=getUser();  
const {id,name}=user  
// const id=user.id,name=user.name
```

上ではuserオブジェクトから`id`プロパティと`name`プロパティを取り出しています。

分割代入（配列）

```
const numbers=[3,1,4,1,5,9,2,6,5]  
const [first,second,third]=numbers  
// const first=numbers[0],second=numbers[1],third=numbers[2];
```

配列の要素を変数として取り出したいときには`let(const) [変数名...]=取り出したい配列`と書けます。右では`numbers`配列から1~3番目の要素を`first`から`third`に取り出しています。

【前提】 immutableについて

すべての変数（インスタンス）はimmutableかどうかを設定されています。immutableなオブジェクトとは作成後にその状態を変えることのできないオブジェクトです。例えば`let n=1`, `const name="Tommy"`など数値型, 文字列型, 真偽型, などは作成した後に変えることができません。

```
let n=1
n++

let name="Tommy"
name[0]="s"

let numbers=[]
numbers.push(2)
console.log(n,name,numbers)
// 2 Tommy numbers
```

- この場合`n`の値は1から2に変化していますが, 内部的には「1」という数値は一度破棄され, 「2」という新しい実態が生成されています。
- `name`はimmutableなのでインデックスで文字を参照することはできますが, 文字を変更することはできません。
- 対して, 配列`numbers`はimmutable(不変)なので`.push(1)`を行った後も同じ実態が維持されています。immutableな値は明示的にnewしない限り新しく作り直されることはありません。

```
let numbers=[]
let newNumbers=numbers
numbers.push(1)
console.log(newNumbers)
// [1]
//
```

`numbers`と`newNumbers`は同じ実態を参照しているので, `numbers`に行った変更が`newNumbers`でも反映されています。`[]`は`new Array()`, `{}`は`new Object()`の特別な書き方です。それぞれ`Array`と`Object`というクラスをインスタンス化しています。

スプレッド構文

iterableオブジェクト

```
// React useStateの例
// todosは配列
setTodos(prev=>{
  const newTodos=[getNewTodo(),...todos];
  return newTodos;
})
```

iterableオブジェクト(≡配列)の要素を1つずつ取り出したい場合に`...変数`と表記します。
`...変数`は`変数[0]`, `変数[1]`, `変数[2]`...と同じです。

配列でないものを明示的に配列でないものを配列にしたい場合や、配列からimmutableに新し配列を生成したい場合に用います。

スプレッド構文があると便利な例

```
const URLs=[...document.querySelectorAll("a")].map(a=>{
  return a.href
})
```

この場合`document.querySelectorAll`の返り値は`NodeList<HTMLAnchorElement>`であり配列ではありません。よってそのままでは`map`が使えませんのでスプレッド構文で配列に変換してから`map`を実行しています。

スプレッド構文（オブジェクト）

```
const todo={content:"勉強する",createdAt:new Date()}
const newTodo={id:crypto.randomUUID(),...todo}
```

オブジェクトのプロパティを列挙したい場合にに使います。上の例では

`{id:crypto.randomUUID(),...todo}`は

`{id:crypto.randomUUID(),content:todo.content,createdAt:todo.createdAt}`と同じです。

モジュール構文（import/export）

JavaScriptのファイルは**モジュール**として分割して管理できます。別のファイルの関数や変数を使いたいときに**import**、外部に公開したいときに**export**を使います。Reactではコンポーネントを別ファイルに分けるために必ず使います。

named export / named import

複数の関数や変数をファイルから公開したいときに使います。

```
// math.js
export const add = (a, b) => a + b;
export const subtract = (a, b) => a - b;
```

```
// main.js
import { add, subtract } from './math.js';
console.log(add(1, 2)); // 3
```

importするときは`{}`の中に公開されている名前をそのまま書きます。

default export / default import

1つのファイルから1つだけ公開したいときに使います。Reactのコンポーネントファイルでよく使われます。

```
// Greeting.jsx
export default function Greeting({ name }) {
  return <h1>Hello, {name}!</h1>;
}
```

```
// App.jsx
import Greeting from './Greeting.jsx';
```

`default export`されたものは`import`するときに好きな名前をつけられます (`{}`は不要)。

配列メソッド

JavaScriptの配列には、要素を変換・絞り込み・検索するための便利なメソッドが用意されています。Reactでリストを表示するときに特によく使います。

map

`map`は配列の各要素に関数を適用し、**新しい配列**を返します。元の配列は変更されません。

```
const numbers = [1, 2, 3, 4, 5];
const doubled = numbers.map(n => n * 2);
console.log(doubled); // [2, 4, 6, 8, 10]
```

```
const users = [
  { id: 1, name: "Alice" },
  { id: 2, name: "Bob" },
];
const names = users.map(user => user.name);
console.log(names); // ["Alice", "Bob"]
```

Reactでは、配列のデータをJSX要素の配列に変換するときに使います。

filter

`filter`は条件を満たす要素だけを抽出した**新しい配列**を返します。

```
const numbers = [1, 2, 3, 4, 5, 6];
const evens = numbers.filter(n => n % 2 === 0);
console.log(evens); // [2, 4, 6]
```

```
const todos = [
  { id: 1, text: "勉強する", done: true },
  { id: 2, text: "買い物する", done: false },
  { id: 3, text: "運動する", done: false },
];
```

```
const remaining = todos.filter(todo => !todo.done);
console.log(remaining);
// [{ id: 2, ... }, { id: 3, ... }]
```

Todoアプリで「完了済みを除く」などの絞り込みに使えます。

find

findは条件を満たす**最初の1つ**の要素を返します。見つからない場合は**undefined**を返します。

```
const users = [
  { id: 1, name: "Alice" },
  { id: 2, name: "Bob" },
  { id: 3, name: "Charlie" },
];
const user = users.find(u => u.id === 2);
console.log(user); // { id: 2, name: "Bob" }
```

filterは複数の要素を返しますが、**find**は1つだけ返す点が違います。IDでデータを1件取得したいときに使います。

非同期処理

Promiseオブジェクト

PromiseオブジェクトとはJavaScriptにおいて非同期処理を行うための仕組みです。Promiseオブジェクトは以下の3つの状態を持ちます。

- pending（保留中）：非同期処理がまだ完了していない状態
- fulfilled（成功）：非同期処理が成功した状態
- rejected（失敗）：非同期処理が失敗した状態 Promiseオブジェクトは以下のようにして作成します。

```
const promise=new Promise((resolve,reject)=>{
  // 非同期処理をここに書く
  // 成功した場合はresolve(value)を呼び出す
  // 失敗した場合はreject(error)を呼び出す
})
```

Promiseオブジェクトは以下のようにして使用します。

```
promise.then(value=>{
  // 非同期処理が成功した場合の処理
}).catch(error=>{
  // 非同期処理が失敗した場合の処理
})
```

async/await構文

async/await構文はPromiseオブジェクトをより簡潔に扱うための構文です。async関数は常にPromiseオブジェクトを返します。awaitキーワードはPromiseオブジェクトが解決されるまで待機します。

```
async function fetchData(){
  try{
    const response=await fetch("https://api.example.com/data");
    const data=await response.json();
    console.log(data);
  }catch(error){
    console.error("Error fetching data:",error);
  }
}
```

fetch()を使ったAPIからのデータ取得

fetch()関数はAPIからデータを取得するための関数です。fetch()関数はPromiseオブジェクトを返します。fetch()関数の引数にはURLを指定します。

```
fetch("https://api.example.com/data")
  .then(response=>response.json())
  .then(data=>{
    console.log(data);
  })
  .catch(error=>{
    console.error("Error fetching data:",error);
  })
```

fetch()関数はHTTPリクエストを送信し、サーバーからのレスポンスを受け取ります。レスポンスはResponseオブジェクトとして返されます。Responseオブジェクトにはjson()メソッドがあり、これを呼び出すとレスポンスの内容をJSON形式で取得できます。

Viteを使ったReactの導入

Viteは高速なフロントエンドビルドツールで、Reactプロジェクトのセットアップを簡単に行うことができます。以下のコマンドを実行してViteを使ったReactプロジェクトを作成します。

```
npm create vite@latest my-react-app -- --template react
```

このコマンドはViteを使用してReactプロジェクトを作成します。**my-react-app**はプロジェクトの名前で、必要に応じて変更してください。プロジェクトが作成されたら、以下のコマンドで依存関係をインストールし、開発サーバーを起動します。

```
cd my-react-app
npm install
npm run dev
```

これでブラウザで<http://localhost:5173>にアクセスすると、Reactアプリケーションが表示されます。

React入門

なぜReactを使うのか（通常のDOM操作との違い）

Reactはユーザーインターフェースを構築するためのJavaScriptライブラリで、通常のDOM操作と比べて以下の利点があります。

- **宣言的なUI**: Reactでは、UIをどのように見せたいかを宣言的に記述できます。これにより、コードがより読みやすく、保守しやすくなります。
- **コンポーネントベース**: ReactはUIを小さな再利用可能なコンポーネントに分割して構築します。これにより、コードの再利用性が高まり、複雑なUIを管理しやすくなります。
- **仮想DOM**: Reactは仮想DOMを使用して、UIの変更を効率的に管理します。これにより、パフォーマンスが向上し、ユーザーエクスペリエンスが向上します。
- **豊富なエコシステム**: Reactには豊富なライブラリやツールが存在し、開発者はこれらを活用して効率的にアプリケーションを構築できます。

JSXの文法と制約

JSXはJavaScriptの拡張構文で、ReactコンポーネントのUIを記述するために使用されます。JSXはHTMLのような構文を持ちますが、JavaScriptのコードとして解釈されます。以下はJSXの基本的な文法と制約です。

- JSXはJavaScriptの式を埋め込むことができます。式は波括弧`{}`で囲む必要があります。

```
const name="Tommy";
const element=<h1>Hello, {name}!</h1>;
```

- JSXでは、HTMLの属性はキャメルケースで記述する必要があります。例えば、`class`は`className`、`for`は`htmlFor`になります。

```
const element=<div className="container">...</div>;
```

- JSXは1つの親要素を持つ必要があります。複数の要素を返す場合は、`React.Fragment`を使用するか、空のタグ`<>...</>`を使用します。

```
const element=(
  <>
    <h1>Title</h1>
    <p>Paragraph</p>
  </>
);
```

- JSXはJavaScriptの式であるため、条件分岐やループなどのロジックを直接記述することはできません。これらのロジックはJavaScriptのコード内で処理し、その結果をJSXに埋め込む必要があります。

```
const isLoggedIn=true;
const element=isLoggedIn ? <h1>Welcome back!</h1> : <h1>Please sign up.</h1>;
```

関数コンポーネントの作り方

関数コンポーネントは、JavaScriptの関数を使用してReactコンポーネントを定義する方法です。関数コンポーネントは、引数としてpropsを受け取り、JSXを返します。以下は関数コンポーネントの例です。

```
function Greeting(props){
  return <h1>Hello, {props.name}!</h1>;
}
```

この例では、**Greeting**という関数コンポーネントが定義されており、**props**を引数として受け取ります。**props.name**を使用して、渡された名前を表示しています。関数コンポーネントは、Reactの最新の機能であるフックと組み合わせて使用されることが多いです。

propsでデータを渡す

propsは、親コンポーネントから子コンポーネントにデータを渡すための仕組みです。propsはオブジェクトとして渡され、子コンポーネントはこのオブジェクトを使用して必要なデータにアクセスします。以下はpropsを使用してデータを渡す例です。

```
function Greeting(props){
  return <h1>Hello, {props.name}!</h1>;
}
function App(){
  return <Greeting name="Tommy" />;
}
```

この例では、**App**コンポーネントが**Greeting**コンポーネントに**name**というpropを渡しています。**Greeting**コンポーネントは、**props.name**を使用して渡された名前を表示しています。

childrenの使いかた

childrenは、コンポーネントの開始タグと終了タグの間に挿入された要素を表す特別なpropです。childrenを使用することで、コンポーネントの中に任意の内容を挿入することができます。以下はchildrenを使用した例です。

```
function Container(props){
  return <div className="container">{props.children}</div>;
}
function App(){
  return (
    <Container>
      <h1>Title</h1>
      <p>Paragraph</p>
    </Container>
  )
}
```

```
);  
}
```

この例では、`Container`コンポーネントが`props.children`を使用して、`App`コンポーネントから渡された内容を表示しています。`App`コンポーネントは、`Container`コンポーネントの中に`<h1>`と`<p>`要素を挿入しています。childrenを使用することで、柔軟なコンポーネントの構造を作成することができます。

React基礎

useState

`useState`はコンポーネント内で 状態 (state) を管理するためのフックです。ボタンのクリック数・入力フォームの内容・APIから取得したデータなど、「変化するデータ」はすべてstateで管理します。

```
import { useState } from 'react';  
  
function Counter() {  
  const [count, setCount] = useState(0);  
  
  return (  
    <div>  
      <p>カウント: {count}</p>  
      <button onClick={() => setCount(count + 1)}>+1</button>  
    </div>  
  );  
}
```

`useState(初期値)`は[現在の値, 値を更新する関数]の配列を返します。分割代入で受け取るのが一般的です。

stateを直接書き換えてはいけません。 必ず`setCount`のような更新関数を使ってください。直接書き換えても画面は再描画されません。

```
// NG  
count = count + 1;  
  
// OK  
setCount(count + 1);
```

イベント処理 (onClick, onChangeなど)

JSXではHTMLのイベント属性に対応する`onClick`・`onChange`などのプロパティに関数を渡すことでイベントを処理します。

onClick

```
function Button() {  
  const handleClick = () => {  
    alert('ボタンがクリックされました');  
  };  
}
```

```
};

return <button onClick={handleClick}>クリック</button>;
}
```

`onClick={handleClick()}`と書くと**即座に実行**されてしまいます。`onClick={handleClick}`のように関数自体を渡すのが正しい書き方です。

onChange (フォーム入力)

`onChange`は入力内容が変わるたびに呼び出されます。`useState`と組み合わせて入力値を管理します。

```
function TextInput() {
  const [text, setText] = useState('');

  return (
    <div>
      <input
        type="text"
        value={text}
        onChange={(e) => setText(e.target.value)}
      />
      <p>入力内容: {text}</p>
    </div>
  );
}
```

`e.target.value`でその時点の入力値を取得できます。`value={text}`を指定することで、stateと入力欄の内容を常に同期させています（これを**controlled component**と呼びます）。

useEffect

`useEffect`はコンポーネントの**副作用**（データの取得・タイマーの設定など）を扱うためのフックです。レンダリングのたびに実行されると困る処理を、適切なタイミングだけ実行するために使います。

```
import { useState, useEffect } from 'react';

function Timer() {
  const [seconds, setSeconds] = useState(0);

  useEffect(() => {
    const id = setInterval(() => {
      setSeconds(s => s + 1);
    }, 1000);

    return () => clearInterval(id); // クリーンアップ
  }, []); // []はコンポーネントのマウント時に1回だけ実行
```

```
return <p>{seconds}秒経過</p>;
}
```

`useEffect(処理, [依存配列])`という形で使います。

依存配列 実行タイミング

なし	すべての再レンダリング後
<code>[]</code>	マウント時（初回表示）に1回だけ
<code>[value]</code>	<code>value</code> が変わるたびに実行

useEffect + fetchでデータの取得

APIからデータを取得してコンポーネントに表示するときは、`useEffect`と`fetch`を組み合わせます。

```
import { useState, useEffect } from 'react';

function UserList() {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const fetchUsers = async () => {
      const response = await fetch('https://jsonplaceholder.typicode.com/users');
      const data = await response.json();
      setUsers(data);
      setLoading(false);
    };

    fetchUsers();
  }, []);

  if (loading) return <p>読み込み中...</p>;

  return (
    <ul>
      {users.map(user => (
        <li key={user.id}>{user.name}</li>
      ))}
    </ul>
  );
}
```

`useEffect`の中で直接`async`関数を使うことはできないため、内側に`async`関数を定義してから呼び出します。

条件付きレンダリング（&& や三項演算子）

条件によって表示する内容を切り替えたいときは、JSXの中でJavaScriptの演算子を使います。

&& (短絡評価)

条件が`true`のときだけ要素を表示したい場合に使います。

```
function Alert({ message }) {
  return (
    <div>
      {message && <p className="alert">{message}</p>}}
    </div>
  );
}
```

`message`が空文字や`null`のときは何も表示されません。

三項演算子

`true`のときと`false`のときで表示を切り替えたい場合に使います。

```
function LoginStatus({ isLoggedIn }) {
  return (
    <div>
      {isLoggedIn ? <p>ログイン中です</p> : <p>ログインしてください</p>}
    </div>
  );
}
```

リストレンダリングとkey属性

配列のデータを元にリストを表示するには、`map`を使ってJSX要素の配列に変換します。

```
function TodoList({ todos }) {
  return (
    <ul>
      {todos.map(todo => (
        <li key={todo.id}>{todo.text}</li>
      ))}
    </ul>
  );
}
```

`key`属性はReactがどの要素が変化・追加・削除されたかを追跡するために必要です。**リスト内の各要素に必ず一意な`key`を指定してください。**

配列のインデックス (`key={i}`) は要素の順番が変わると正しく動作しないことがあるため、IDなど変化しない値を使うのが望ましいです。

CSSモジュール

CSSモジュールを使うと、CSSのクラス名が自動的にそのコンポーネントだけにスコープされます。別のコンポーネントで同じクラス名を使っても衝突しないため、大きなプロジェクトでも安全にスタイルを管理できます。

使い方

ファイル名をコンポーネント名.module.cssにします。

```
/* Button.module.css */
.button {
  background-color: #4c8bf5;
  color: white;
  padding: 8px 16px;
  border: none;
  border-radius: 4px;
  cursor: pointer;
}

.button:hover {
  background-color: #3a7ae0;
}
```

コンポーネントファイルでimportして使います。

```
// Button.jsx
import styles from './Button.module.css';

function Button({ label }) {
  return <button className={styles.button}>{label}</button>;
}

export default Button;
```

styles.buttonのように、stylesオブジェクトのプロパティとしてクラス名にアクセスします。ブラウザ上ではbutton_button__xxxxのようなユニークな名前に変換されるため、他のコンポーネントのスタイルと衝突しません。